

Heurísticas y metaheurísticas

Algoritmos y Estructuras de Datos III

Metaheurísticas

- ▶ Heurísticas clásicas.
- ▶ Metaheurísticas o heurísticas “modernas”.

¿Cuándo usarlas?

- ▶ Problemas para los cuales no se conocen buenos algoritmos exactos.
- ▶ Problemas difíciles de modelar.

¿Cómo se evalúan?

- ▶ Problemas test.
- ▶ Problemas reales.
- ▶ Problemas generados al azar.
- ▶ Cotas inferiores.

Problema del viajante de comercio (TSP)

Definición: Dado un grafo $G = (V, X)$ con longitudes asignadas a las aristas, $l : X \rightarrow \mathbb{R}^{\geq 0}$, queremos determinar un circuito hamiltoniano de longitud mínima.

- ▶ No se conocen algoritmos polinomiales para resolver el problema del viajante de comercio.
- ▶ Tampoco se conocen algoritmos ϵ -aproximados polinomiales para el TSP general (si se conocen cuando las distancias son euclidianas).
- ▶ Es el problema de optimización combinatoria más estudiado.

Heurísticas y algoritmos aproximados para el TSP

Heurística del vecino más cercano

```
elegir un nodo  $v$   
 $orden(v) := 0$   
 $S := \{v\}$   
 $i := 0$   
mientras  $S \neq V$  hacer  
     $i := i + 1$   
    elegir la arista  $(v, w)$  más barata con  $w \notin S$   
     $orden(w) := i$   
     $S := S \cup \{w\}$   
     $v := w$   
fin mientras  
retornar  $orden$ 
```

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

```
C := un circuito de longitud 3
S := {nodos de C}
mientras  $S \neq V$  hacer
    ELEGIR un nodo  $v \notin S$ 
     $S := S \cup \{v\}$ 
    INSERTAR  $v$  en  $C$ 
fin mientras
retornar  $C$ 
```

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

```
C := un circuito de longitud 3
S := {nodos de C}
mientras  $S \neq V$  hacer
    ELEGIR un nodo  $v \notin S$ 
     $S := S \cup \{v\}$ 
    INSERTAR  $v$  en  $C$ 
fin mientras
retornar  $C$ 
```

¿Cómo ELEGIR?

¿Cómo INSERTAR?

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

```
C := un circuito de longitud 3  
S := {nodos de C}  
mientras  $S \neq V$  hacer  
    ELEGIR un nodo  $v \notin S$   
     $S := S \cup \{v\}$   
    INSERTAR  $v$  en  $C$   
fin mientras  
retornar  $C$ 
```

¿Cómo ELEGIR?

↪ **variantes de la heurística de inserción**

¿Cómo INSERTAR?

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

Para INSERTAR el nodo v elegido:

- ▶ Sea $c_{v_i v_j}$ es el costo o la longitud de la arista (v_i, v_j) .
- ▶ Elegimos dos nodos consecutivos en el circuito v_i, v_{i+1} tal que

$$c_{v_i v} + c_{v v_{i+1}} - c_{v_i v_{i+1}}$$

sea mínimo.

- ▶ Insertamos v entre v_i y v_{i+1} .

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

Podemos ELEGIR el nuevo nodo v para agregar al circuito tal que:

- ▶ v sea el nodo más próximo a un nodo que ya está en el circuito.

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

Podemos ELEGIR el nuevo nodo v para agregar al circuito tal que:

- ▶ v sea el nodo más próximo a un nodo que ya está en el circuito.
- ▶ v sea el nodo más lejano a un nodo que ya está en el circuito.

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

Podemos ELEGIR el nuevo nodo v para agregar al circuito tal que:

- ▶ v sea el nodo más próximo a un nodo que ya está en el circuito.
- ▶ v sea el nodo más lejano a un nodo que ya está en el circuito.
- ▶ v sea el nodo *más barato*, o sea el que hace crecer menos la longitud del circuito.

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

Podemos ELEGIR el nuevo nodo v para agregar al circuito tal que:

- ▶ v sea el nodo más próximo a un nodo que ya está en el circuito.
- ▶ v sea el nodo más lejano a un nodo que ya está en el circuito.
- ▶ v sea el nodo *más barato*, o sea el que hace crecer menos la longitud del circuito.
- ▶ v se elige al azar.

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de inserción

En el caso de grafos euclidianos (por ejemplo grafos en el plano $\mathbb{R}^2$), se puede implementar un algoritmo de inserción:

- ▶ Usando la cápsula convexa de los nodos como circuito inicial.
- ▶ Insertando en cada paso un nodo v tal que el ángulo formado por las aristas (w, v) y (v, z) , con w y z consecutivos en el circuito ya construido, sea máximo.

Hay muchas variantes sobre estas ideas.

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de mejoramiento - Algoritmos de búsqueda local

¿Cómo podemos mejorar la solución obtenida por alguna heurística constructiva como las anteriores?

Heurística 2-opt de Lin y Kernighan

obtener una solución inicial H

por ejemplo con alguna de las heurísticas anteriores

mientras sea posible **hacer**

elegir (u_i, u_{i+1}) y $(u_k, u_{k+1}) \in H$ tal que

$$c_{u_i u_{i+1}} + c_{u_k u_{k+1}} > c_{u_i u_k} + c_{u_{i+1} u_{k+1}}$$

$$H := H \setminus \{(u_i, u_{i+1}), (u_k, u_{k+1})\} \cup \{(u_i, u_k), (u_{i+1}, u_{k+1})\}$$

fin mientras

¿Cuándo para este algoritmo? ¿Se obtiene la solución óptima del TSP de este modo?

Heurísticas y algoritmos aproximados para el TSP

Heurísticas de mejoramiento - Algoritmos de búsqueda local

- ▶ En vez de elegir para sacar de H un par de aristas cualquiera que nos lleve a obtener un circuito de menor longitud podemos elegir, entre todos los pares posibles, el par que nos hace obtener el menor circuito (más trabajo computacional).
- ▶ Esta idea se extiende en las heurísticas k -opt donde se hacen intercambios de k aristas. Es decir, en vez de sacar dos aristas, sacamos k aristas de H y vemos cual es la mejor forma de reconstruir el circuito. En la práctica se usa sólo 2-opt o 3-opt.

Algoritmos de descenso o búsqueda local

Esquema general

S = conjunto de soluciones

$N(s)$ = soluciones “vecinas” de la solución s

$f(s)$ = valor de la solución s

elegir una solución inicial $s^* \in S$

repetir

 elegir $s \in N(s^*)$ tal que $f(s) < f(s^*)$

 reemplazar s^* por s

hasta que $f(s) > f(s^*)$ para todos los $s \in N(s^*)$

Algoritmos de descenso o búsqueda local

- ▶ ¿Cómo determinar las soluciones vecinas de una solución s dada?

Algoritmos de descenso o búsqueda local

- ▶ ¿Cómo determinar las soluciones vecinas de una solución s dada?
- ▶ ¿Qué se obtiene con este procedimiento? ¿Sirve?

Algoritmos de descenso o búsqueda local

- ▶ ¿Cómo determinar las soluciones vecinas de una solución s dada?
- ▶ ¿Qué se obtiene con este procedimiento? ¿Sirve?
- ▶ Óptimos locales y globales

Algoritmos de descenso o búsqueda local

- ▶ ¿Cómo determinar las soluciones vecinas de una solución s dada?
- ▶ ¿Qué se obtiene con este procedimiento? ¿Sirve?
- ▶ Óptimos locales y globales
- ▶ Espacio de búsqueda

Algoritmos de descenso o búsqueda local

Ejemplo

- ▶ Tenemos que asignar n tareas a un sola máquina.
- ▶ Cada trabajo j tiene un tiempo de procesamiento p_j y una fecha prometida de entrega d_j .
- ▶ El objetivo es minimizar

$$T = \sum_{j=1}^n \max\{(C_j - d_j), 0\}$$

donde C_j es el momento en que se completa el trabajo j .

Algoritmos de descenso o búsqueda local

Ejemplo

- ▶ **Cómo elegir las soluciones iniciales:** Se podría tomar cualquier permutación de las tareas.
- ▶ **Determinación de los vecinos de una solución dada:** Podemos tomar las que se obtengan de la solución actual cambiando la posición de un trabajo con otro.

Algoritmos de descenso o búsqueda local

Ejemplo

- ▶ **Cómo elegir las soluciones iniciales:** Se podría tomar cualquier permutación de las tareas.
- ▶ **Determinación de los vecinos de una solución dada:**
Podemos tomar las que se obtengan de la solución actual cambiando la posición de un trabajo con otro.

En un problema con 4 trabajos, los vecinos de $s = (1, 2, 3, 4)$ serán:

$$N(s) = \{(1, 3, 2, 4), (3, 2, 1, 4), (1, 2, 4, 3), (1, 4, 3, 2), (2, 1, 3, 4), (4, 2, 3, 1)\}$$

GRASP

Esquema general

Feo, T., Resende, M., “Greedy randomized adaptive search procedures”, Journal of Global Optimization, 1995, pp 1, 27.

```
mientras no se verifique el criterio de parada hacer  
    solución:=construirSoluciónGRA  
    mejorSolución:=búsquedaLocal(solución)  
    actualizarSolución(solución,mejorSolución)
```


GRASP

Esquema general

- ▶ **Algoritmo construirSoluciónGRA:** En vez de usar un algoritmo goloso que elija el elemento más prometedor, según una función “adaptativa”, para agregar a la solución, en cada iteración se elige al azar entre los que cumplen que no pasan de un porcentaje α del valor del mejor elemento. Se puede limitar el tamaño de la lista de estos elementos.
- ▶ **Algoritmo búsquedaLocal:** Definición de intercambios.